```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;



entity Adder is
    generic (
            N : natural := 64
    );
    Port (
        A    : in  std_logic_vector(N-1 downto 0);
        B    : in  std_logic_vector(N-1 downto 0);
        Cin  : in  std_logic;
        S    : out std_logic_vector(N-1 downto 0);
        Cout : out std_logic;
        Ovfl : out std_logic
    );
end Adder;

architecture Baseline of Adder is

    signal C : std_logic_vector(N downto 0);
begin

    C(0) <= Cin;


    gen_ripple : for i in 0 to N-1 generate

        S(i) <= A(i) xor B(i) xor C(i);


        C(i+1) <= (A(i) and B(i)) or
                  (A(i) and C(i)) or
                  (B(i) and C(i));
    end generate;


    Cout <= C(N);


    Ovfl <= C(N) xor C(N-1);
end architecture Baseline;



architecture Fastripple of Adder is
    signal A_u, B_u : unsigned(N downto 0);
    signal Cin_u    : unsigned(N downto 0);
    signal result   : unsigned(N downto 0);
begin

    A_u <= resize(unsigned(A), N+1);
    B_u <= resize(unsigned(B), N+1);

     Cin_u <= (others => '0') when Cin='0'
         else to_unsigned(1, N+1);

     result <= A_u + B_u + Cin_u;

     S    <= std_logic_vector(result(N-1 downto 0));
     Cout <= result(N);


     Ovfl <= (not (A(N-1) xor B(N-1))) and
              (std_logic(result(N-1)) xor A(N-1));



```

```
70  end architecture Fastripple;
71
72
73
74  architecture ConditionalSum of Adder is
75
76      signal sum_low  : std_logic_vector(N/2-1 downto 0);
77      signal sum_high0, sum_high1 : std_logic_vector(N/2-1 downto 0);
78
79      signal carry_mid : std_logic;
80      signal cout_high0, cout_high1 : std_logic;
81      signal cout_int  : std_logic;
82
83      signal S_int : std_logic_vector(N-1 downto 0);
84
85  begin
86
87
88      base_case: if N = 1 generate
89          base: entity work.Adder(Baseline)
90          generic map (N => 1)
91          port map (
92              A => A,
93              B => B,
94              Cin => Cin,
95              S => S,
96              Cout => Cout,
97              Ovfl => Ovfl
98          );
99      end generate;
100
101
102     recursive_case: if N > 1 generate
103
104
105         lower: entity work.Adder(ConditionalSum)
106         generic map (N => N/2)
107         port map (
108             A => A(N/2-1 downto 0),
109             B => B(N/2-1 downto 0),
110             Cin => Cin,
111             S => sum_low,
112             Cout => carry_mid,
113             Ovfl => open
114         );
115
116
117         upper0: entity work.Adder(ConditionalSum)
118         generic map (N => N/2)
119         port map (
120             A => A(N-1 downto N/2),
121             B => B(N-1 downto N/2),
122             Cin => '0',
123             S => sum_high0,
124             Cout => cout_high0,
125             Ovfl => open
126         );
127
128
129         upper1: entity work.Adder(ConditionalSum)
130         generic map (N => N/2)
131         port map (
132             A => A(N-1 downto N/2),
133             B => B(N-1 downto N/2),
134             Cin => '1',
135             S => sum_high1,
136             Cout => cout_high1,
137             Ovfl => open
138         );
```

```
139
140
141           S_int(N/2-1 downto 0) <= sum_low;
142
143           S_int(N-1 downto N/2) <= sum_high0 when carry_mid='0'
144                                    else sum_high1;
145
146           cout_int <= cout_high0 when carry_mid='0'
147                       else cout_high1;
148
149           S <= S_int;
150           Cout <= cout_int;
151
152
153           Ovfl <= (not (A(N-1) xor B(N-1)))
154                   and (S_int(N-1) xor A(N-1));
155
156       end generate;
157
158   end architecture ConditionalSum;
159
160   architecture BCLA of Adder is
161
162
163      signal P, G : std_logic_vector(15 downto 0);
164      signal C    : std_logic_vector(16 downto 0);
165
166   begin
167
168      C(0) <= Cin;
169
170
171      gen_blocks: for i in 0 to 15 generate
172          signal p_bit, g_bit : std_logic_vector(3 downto 0);
173          signal c_int        : std_logic_vector(4 downto 0);
174      begin
175
176          c_int(0) <= C(i);
177
178          -
179          p_bit <= A(4*i+3 downto 4*i) xor B(4*i+3 downto 4*i);
180          g_bit <= A(4*i+3 downto 4*i) and B(4*i+3 downto 4*i);
181
182
183          c_int(1) <= g_bit(0) or (p_bit(0) and c_int(0));
184
185          c_int(2) <= g_bit(1) or
186                      (p_bit(1) and g_bit(0)) or
187                      (p_bit(1) and p_bit(0) and c_int(0));
188
189          c_int(3) <= g_bit(2) or
190                      (p_bit(2) and g_bit(1)) or
191                      (p_bit(2) and p_bit(1) and g_bit(0)) or
192                      (p_bit(2) and p_bit(1) and p_bit(0) and c_int(0));
193
194          c_int(4) <= g_bit(3) or
195                      (p_bit(3) and g_bit(2)) or
196                      (p_bit(3) and p_bit(2) and g_bit(1)) or
197                      (p_bit(3) and p_bit(2) and p_bit(1) and g_bit(0)) or
198                      (p_bit(3) and p_bit(2) and p_bit(1) and p_bit(0) and c_int(0));
199
200
201          S(4*i+3 downto 4*i) <= p_bit xor c_int(3 downto 0);
202
203
204          P(i) <= p_bit(3) and p_bit(2) and p_bit(1) and p_bit(0);
205
206          G(i) <= g_bit(3) or
207                  (p_bit(3) and g_bit(2)) or
```

```
                (p_bit(3) and p_bit(2) and g_bit(1)) or
                (p_bit(3) and p_bit(2) and p_bit(1) and g_bit(0));

    end generate;


    C(1) <= G(0) or (P(0) and C(0));

    C(2) <= G(1) or
            (P(1) and G(0)) or
            (P(1) and P(0) and C(0));

    C(3) <= G(2) or
            (P(2) and G(1)) or
            (P(2) and P(1) and G(0)) or
            (P(2) and P(1) and P(0) and C(0));



    gen_carries: for i in 3 to 15 generate
    begin
        C(i+1) <= G(i) or (P(i) and C(i));
    end generate;


    Cout <= C(16);

    OVFL <= C(15) xor C(16);

end BCLA;

architecture CarrySelect of Adder is


     signal S0_1, S1_1 : std_logic_vector(3  downto 0);
     signal C0_1, C1_1 : std_logic;


     signal S0_2, S1_2 : std_logic_vector(7  downto 0);
     signal C0_2, C1_2 : std_logic;


     signal S0_3, S1_3 : std_logic_vector(7  downto 0);
     signal C0_3, C1_3 : std_logic;


     signal S0_4, S1_4 : std_logic_vector(15 downto 0);
     signal C0_4, C1_4 : std_logic;


     signal S0_5, S1_5 : std_logic_vector(15 downto 0);
     signal C0_5, C1_5 : std_logic;


     signal S0_6, S1_6 : std_logic_vector(7  downto 0);
     signal C0_6, C1_6 : std_logic;
     signal OV0_6, OV1_6 : std_logic;

     signal Csel : std_logic_vector(6 downto 0);

begin


     blk0: entity work.Adder(Baseline)
         generic map (N => 4)
         port map (
             A    => A(3 downto 0),
             B    => B(3 downto 0),
             Cin  => Cin,
```

```
277             S    => S(3 downto 0),
278             Cout => Csel(0),
279             Ovfl => open
280         );
281
282
283     blk1_c0: entity work.Adder(Baseline)
284         generic map (N => 4)
285         port map (
286                 A => A(7 downto 4),
287                 B => B(7 downto 4),
288             Cin => '0',
289                 S => S0_1,
290                 Cout => C0_1,
291                 Ovfl => open
292           );
293
294     blk1_c1: entity work.Adder(Baseline)
295         generic map (N => 4)
296         port map (
297                 A => A(7 downto 4),
298                 B => B(7 downto 4),
299             Cin => '1',
300                 S => S1_1,
301                 Cout => C1_1,
302                 Ovfl => open
303         );
304
305     S(7  downto 4)  <= S1_1 when Csel(0) = '1' else S0_1;
306     Csel(1)         <= C1_1 when Csel(0) = '1' else C0_1;
307
308
309     blk2_c0: entity work.Adder(Baseline)
310         generic map (N => 8)
311         port map (
312               A => A(15 downto 8),
313               B => B(15 downto 8),
314            Cin => '0',
315               S => S0_2,
316               Cout => C0_2,
317               Ovfl => open
318         );
319
320     blk2_c1: entity work.Adder(Baseline)
321         generic map (N => 8)
322         port map (
323               A => A(15 downto 8),
324               B => B(15 downto 8),
325               Cin => '1',
326               S => S1_2,
327               Cout => C1_2,
328               Ovfl => open
329         );
330
331     S(15 downto 8)  <= S1_2 when Csel(1) = '1' else S0_2;
332     Csel(2)         <= C1_2 when Csel(1) = '1' else C0_2;
333
334     blk3_c0: entity work.Adder(Baseline)
335         generic map (N => 8)
336         port map (
337               A => A(23 downto 16),
338               B => B(23 downto 16),
339               Cin => '0',
340               S => S0_3,
341               Cout => C0_3,
342               Ovfl => open
343         );
344
345     blk3_c1: entity work.Adder(Baseline)
```

```
346         generic map (N => 8)
347         port map (
348               A => A(23 downto 16),
349               B => B(23 downto 16),
350               Cin => '1',
351               S => S1_3,
352               Cout => C1_3,
353               Ovfl => open
354         );
355
356       S(23 downto 16) <= S1_3 when Csel(2) = '1' else S0_3;
357       Csel(3)         <= C1_3 when Csel(2) = '1' else C0_3;
358
359       blk4_c0: entity work.Adder(Baseline)
360           generic map (N => 16)
361                 port map (
362                 A => A(39 downto 24),
363                 B => B(39 downto 24),
364                 Cin => '0',
365                 S => S0_4,
366                 Cout => C0_4,
367                 Ovfl => open
368            );
369
370       blk4_c1: entity work.Adder(Baseline)
371           generic map (N => 16)
372                 port map (
373                 A => A(39 downto 24),
374                 B => B(39 downto 24),
375                 Cin => '1',
376                 S => S1_4,
377                 Cout => C1_4,
378                 Ovfl => open
379           );
380
381       S(39 downto 24) <= S1_4 when Csel(3) = '1' else S0_4;
382       Csel(4)         <= C1_4 when Csel(3) = '1' else C0_4;
383
384       blk5_c0: entity work.Adder(Baseline)
385           generic map (N => 16)
386           port map (
387                 A => A(55 downto 40),
388                 B => B(55 downto 40),
389                 Cin => '0',
390                 S => S0_5,
391                 Cout => C0_5,
392                 Ovfl => open
393           );
394
395       blk5_c1: entity work.Adder(Baseline)
396           generic map (N => 16)
397           port map (
398                 A => A(55 downto 40),
399                 B => B(55 downto 40),
400                 Cin => '1',
401                 S => S1_5,
402                 Cout => C1_5,
403                 Ovfl => open
404           );
405
406       S(55 downto 40) <= S1_5 when Csel(4) = '1' else S0_5;
407       Csel(5)         <= C1_5 when Csel(4) = '1' else C0_5;
408
409
410       blk6_c0: entity work.Adder(Baseline)
411           generic map (N => 8)
412                 port map (
413                 A => A(63 downto 56),
414                 B => B(63 downto 56),
```

```
                Cin => '0',
                S => S0_6,
                Cout => C0_6,
                Ovfl => OV0_6
        );

    blk6_c1: entity work.Adder(Baseline)
        generic map (N => 8)
              port map (
              A => A(63 downto 56),
              B => B(63 downto 56),
              Cin => '1',
              S => S1_6,
              Cout => C1_6,
              Ovfl => OV1_6
      );

    S(63 downto 56) <= S1_6 when Csel(5) = '1' else S0_6;
    Csel(6)         <= C1_6 when Csel(5) = '1' else C0_6;


    Cout <= Csel(6);
    Ovfl <= Csel(5) xor Csel(6);

end CarrySelect;

architecture CarrySkip of Adder is


    constant BLOCK_SIZE  : positive := 4;
    constant NUM_BLOCKS  : positive := N / BLOCK_SIZE;


    signal c_chain : std_logic_vector(NUM_BLOCKS downto 0);


    signal S_int : std_logic_vector(N-1 downto 0);

begin

    c_chain(0) <= Cin;


    gen_blocks : for i in 0 to NUM_BLOCKS-1 generate
        signal block_prop : std_logic;
        signal ripple_cout : std_logic;
        signal block_a, block_b : std_logic_vector(BLOCK_SIZE-1 downto 0);
    begin


        block_a <= A(BLOCK_SIZE*(i+1)-1 downto BLOCK_SIZE*i);
        block_b <= B(BLOCK_SIZE*(i+1)-1 downto BLOCK_SIZE*i);

        RCA_BLOCK : entity work.Adder(Baseline)
            generic map (N => BLOCK_SIZE)
            port map (
                A    => block_a,
                B    => block_b,
                Cin  => c_chain(i),
                S    => S_int(BLOCK_SIZE*(i+1)-1 downto BLOCK_SIZE*i),
                Cout => ripple_cout,
                Ovfl => open
            );



        block_prop <= (block_a(0) xor block_b(0)) and
                      (block_a(1) xor block_b(1)) and
                      (block_a(2) xor block_b(2)) and
```

```
                        (block_a(3) xor block_b(3));


            c_chain(i+1) <= (block_prop and c_chain(i)) or (not(block_prop) and ripple_cout);

        end generate;


        S    <= S_int;
        Cout <= c_chain(NUM_BLOCKS);


        Ovfl <= (A(N-1) and B(N-1) and not S_int(N-1)) or
                (not A(N-1) and not B(N-1) and S_int(N-1));

    end architecture CarrySkip;

    architecture OCPC of Adder is


        type pg4  is array (0 to 3) of std_logic_vector(1 downto 0);
        type sum8 is array (0 to 7) of std_logic_vector(7 downto 0);


        signal C : std_logic_vector(8 downto 0);


        signal S0 : sum8;
        signal S1 : sum8;

        signal Cout0 : std_logic_vector(7 downto 1);
        signal Cout1 : std_logic_vector(7 downto 1);


        procedure bcla_8 (
            signal   A_in   : in  std_logic_vector(7 downto 0);
            signal   B_in   : in  std_logic_vector(7 downto 0);
            constant cin    : in  std_logic;
            signal   S_out  : out std_logic_vector(7 downto 0);
            signal   Co_out : out std_logic
        ) is
            variable p  : std_logic_vector(3 downto 0);
            variable g  : std_logic_vector(3 downto 0);
            variable pb : std_logic_vector(7 downto 0);
            variable gb : std_logic_vector(7 downto 0);
            variable ci : std_logic_vector(4 downto 0);
        begin

            pb := A_in xor B_in;
            gb := A_in and B_in;

            for k in 0 to 3 loop
                p(k) := pb(2*k+1) and pb(2*k);
                g(k) := gb(2*k+1) or (pb(2*k+1) and gb(2*k));
            end loop;

            ci(0) := cin;
            for k in 0 to 3 loop
                ci(k+1) := g(k) or (p(k) and ci(k));
            end loop;


            for k in 0 to 3 loop

                S_out(2*k)   <= pb(2*k)   xor ci(k);
                S_out(2*k+1) <= pb(2*k+1) xor (gb(2*k) or (pb(2*k) and ci(k)));
            end loop;

            Co_out <= ci(4);
```

```
553         end procedure;
554 
555     begin
556         C(0) <= Cin;
557 
558 
559         process(A, B, Cin)
560             variable p  : std_logic_vector(3 downto 0);
561             variable g  : std_logic_vector(3 downto 0);
562             variable pb : std_logic_vector(7 downto 0);
563             variable gb : std_logic_vector(7 downto 0);
564             variable ci : std_logic_vector(4 downto 0);
565         begin
566             pb := A(7 downto 0) xor B(7 downto 0);
567             gb := A(7 downto 0) and B(7 downto 0);
568             for k in 0 to 3 loop
569                 p(k) := pb(2*k+1) and pb(2*k);
570                 g(k) := gb(2*k+1) or (pb(2*k+1) and gb(2*k));
571             end loop;
572             ci(0) := Cin;
573             for k in 0 to 3 loop
574                 ci(k+1) := g(k) or (p(k) and ci(k));
575             end loop;
576             for k in 0 to 3 loop
577                 S0(0)(2*k)   <= pb(2*k)   xor ci(k);
578                 S0(0)(2*k+1) <= pb(2*k+1) xor (gb(2*k) or (pb(2*k) and ci(k)));
579             end loop;
580             C(1) <= ci(4);
581         end process;
582 
583         S(7 downto 0) <= S0(0);
584 
585         gen_csa : for i in 1 to 7 generate
586 
587             process(A, B)
588                 variable p  : std_logic_vector(3 downto 0);
589                 variable g  : std_logic_vector(3 downto 0);
590                 variable pb : std_logic_vector(7 downto 0);
591                 variable gb : std_logic_vector(7 downto 0);
592                 variable ci : std_logic_vector(4 downto 0);
593             begin
594                 pb := A(8*i+7 downto 8*i) xor B(8*i+7 downto 8*i);
595                 gb := A(8*i+7 downto 8*i) and B(8*i+7 downto 8*i);
596                 for k in 0 to 3 loop
597                     p(k) := pb(2*k+1) and pb(2*k);
598                     g(k) := gb(2*k+1) or (pb(2*k+1) and gb(2*k));
599                 end loop;
600 
601                 ci(0) := '0';
602                 for k in 0 to 3 loop
603                     ci(k+1) := g(k) or (p(k) and ci(k));
604                 end loop;
605                 for k in 0 to 3 loop
606                     S0(i)(2*k)   <= pb(2*k)   xor ci(k);
607                     S0(i)(2*k+1) <= pb(2*k+1) xor (gb(2*k) or (pb(2*k) and ci(k)));
608                 end loop;
609                 Cout0(i) <= ci(4);
610             end process;
611 
612             process(A, B)
613                 variable p  : std_logic_vector(3 downto 0);
614                 variable g  : std_logic_vector(3 downto 0);
615                 variable pb : std_logic_vector(7 downto 0);
616                 variable gb : std_logic_vector(7 downto 0);
617                 variable ci : std_logic_vector(4 downto 0);
618             begin
619                 pb := A(8*i+7 downto 8*i) xor B(8*i+7 downto 8*i);
620                 gb := A(8*i+7 downto 8*i) and B(8*i+7 downto 8*i);
621                 for k in 0 to 3 loop
```

```
622                 p(k) := pb(2*k+1) and pb(2*k);
623                 g(k) := gb(2*k+1) or (pb(2*k+1) and gb(2*k));
624             end loop;
625             -- cin = 1
626             ci(0) := '1';
627             for k in 0 to 3 loop
628                 ci(k+1) := g(k) or (p(k) and ci(k));
629             end loop;
630             for k in 0 to 3 loop
631                 S1(i)(2*k)   <= pb(2*k)   xor ci(k);
632                 S1(i)(2*k+1) <= pb(2*k+1) xor (gb(2*k) or (pb(2*k) and ci(k)));
633             end loop;
634             Cout1(i) <= ci(4);
635         end process;
636
637
638         S(8*i+7 downto 8*i) <= S1(i) when C(i) = '1' else S0(i);
639         C(i+1) <= Cout1(i)   when C(i) = '1' else Cout0(i);
640
641     end generate gen_csa;
642
643     Cout <= C(8);
644     OVFL <= C(7) xor C(8);
645
646 end architecture OCPC;
```